Compiler Design CoSc4212_

June 15, 2025

Compiled by: Natan Asrat

https://www.linkedin.com/in/natan_asrat

Compiler Design

- Compiler
  - a program that reads a program written in one language and translates it into an equivalent program in another language
  - converts human readable instructions to computer readable instructions one time

Compiler Design

- Compiler
  - Applications:
    - Parsers for HTML in web browser
    - Machine code generation for high level languages
    - Software testing
    - Program optimization
    - Malicious code detection
    - Design of new computer architectures

Compiler Design

- Compiler
  - Cousins
    - Preprocessor:
      - produces input for compiler
      - file inclusion, language extension, etc.
    - Assembler
      - assembly language into machine code
      - output of an assembler is called an object file
    - Linker
      - links and merges various object files to make an executable file.
      - determine the memory location where these codes will be loaded

Compiler Design

- Compiler
  - Cousins
    - Loader
      - loading executable files into memory and execute them.
      - calculates size of a program (instructions and data) and creates memory space for it.
      - initializes various registers to initiate execution.
    - Cross-Compiler
      - compiler that runs on one platform and generates executable code for another platform.
    - Source-to-source Compiler
      - compiler that translates source code of one programming language to another

Compiler Design

- Compiler
  - Phases - Analysis: Machine Independent, Language Dependent
    - Lexical / Linear Analysis (scanning)
      - Scans the source code as a stream of characters
      - Represent lexemes in the form of tokens as: <token-name, attribute-value>
      - Token: smallest meaningful element that a compiler understands.
        - Identifiers, Keywords, Literals, Operators and Special symbols.
      - Blanks, new lines, comments will be removed from the source program.

Compiler Design

- Compiler
  - Phases - Analysis: Machine Independent, Language Dependent
    - Syntax / Hierarchical Analysis – Parsing
      - Tokens are grouped hierarchically into nested collections with collective meaning.
      - The result is generally a parse tree.
      - Most syntactic errors in the source program are caught in this phase.
      - Syntactic rules of the source language are given via a Grammar.

Compiler Design

- Compiler
  - Phases - Analysis: Machine Independent, Language Dependent
    - Semantic Analysis
      - make sure that the components of the program fit together meaningfully.
      - checks for semantic errors in the source program (e.g. type mismatch)
      - stores type information in the symbol table or the syntax tree.
        - Types of variables, function parameters, array dimensions, etc.

Compiler Design

- Compiler
  - Phases - Analysis: Machine Independent, Language Dependent
    - Intermediate Code Generation
      - Easy to produce and easy to translate to machine code

Compiler Design

- Compiler
  - Phases - Synthesis: Machine Dependent, Language independent
    - Code Optimization
      - Changes the Intermediate Code by removing inefficiencies
    - Code Generation
      - Converts intermediate code to machine code.
      - handle all aspects of machine architecture
      - Storage allocation decisions are made
      - Register allocation and assignment

Compiler Design

- Interpreter
  - Converts human instructions to machine instructions each time the program is run

Compiler Design

- Lexical Analysis
  - first phase of a compiler
  - input: high level language program
  - output: sequence of tokens
  - Token: string of characters which logically belong together
    - Classes:
      - Identifiers: names chosen by the programmer
      - Keywords: names already in the programming language
      - Separators: punctuation characters
      - Operators: symbols that operate on arguments and produce results
      - Literals: numeric, textual literals
  - Pattern: rule which describes a token
  - Lexeme: sequence of characters matched by a pattern to form the token

Compiler Design

- Syntax Analysis
  - input: streams of tokens from lexical analyzer
  - output: parse tree
  - Parsers or syntax analyzers are generated for a particular grammar
    - CFG are used for syntax specification

Compiler Design

- Syntax Analysis
  - Derivations:
    - Leftmost Derivation
      - Apply a production only to the leftmost variable (non terminal) at every step
      - Example grammar: S → aAS | a | SS, A → SbA | ba
      - S => aAS => aSbAS
      - aSbAS =>aabAS
      - aabAS => aabbaS => aabbaa
    - Rightmost Derivation
      - Apply production to the rightmost variable at every step
      - Example grammar: S → aAS | a | SS, A → SbA | ba
      - S => aAS => aAa
      - aAa=>aSbAa
      - aSbAa =>aSbbaa …

Compiler Design

- Syntax Analysis
  - Parsing: constructing parse tree for a sentence generated by a given grammar.
    - Top Down
      - Starts from the start symbol and transform it to the input
      - LL(1) Grammar (Left-to-right Left-most-derivation 1-lookahead)
    - Bottom Up
      - Starts with the input symbols and tries to construct the parse tree up to the start symbol.
      - LR(k) Parsing (Left-to-right Right-derivation k-lookahead)
        - Types
          - LR(0)
          - SLR(1)
          - LALR(1)
          - CLR(1)

Compiler Design

- Semantic Analysis
  - Syntax Directed Translation: Attaching actions to the grammar rules
    - Actions are executed during the compilation, not during the generation of the compiler

Compiler Design

- Semantic Analysis
  - Syntax Directed Definitions
    - generalization of a context free grammar
    - CFG with attributes and rules

Production	Semantic Rules
L → E return	print(E.val)
E → E1 + T	E.val = E1.val + T.val
E → T	E.val = T.val
T → T1 * F	T.val = T1.val * F.val
T → F	T.val = F.val
T → (E)	F.val = E.val
F → digit	F.val = digit.lexval

Compiler Design

- Semantic Analysis
  - Syntax Directed Definitions - Functions
    - mknode (op, left, right): Creates an operator node with label op & two pointers
    - mkleaf(id, entry), mkleaf(num, val): entry is pointer for identifier i.e. variable ‘a’, val is an actually value like 4

Compiler Design

- Type Checking
  - Types: describes the values to be computed
  - Type Errors: improper operations
  - Type Safety: absence of type errors
  - Type Checking: semantic checks to enforce the type safety
    - Static – done during compilation
      - incompatible operands
      - Flow Control Check
      - Uniqueness Check
      - Name Related Check
    - Dynamic – done during run-time
  - Type Checking Expressions:
    - E → E1 mod E2 { E.type=if E1.type=int and E2.type=int then int else type_error }

Compiler Design

- Intermediate Code Generation
  - Three Address Code
    - sequence of statements of the form
      - X = Y op Z
      - X,Y and Z are names, constants or compiler generated temporaries
      - Op is operator (arithmetic, logical)
    - LHS is the target (X in ‘X = Y op Z’)
    - RHS has at most two sources and one operator (’Y op Z’ in ‘X = Y op Z’)

Compiler Design

- Intermediate Code Generation
  - Implementations
    - Quadruples: instruction is divided into four fields
      - Operator, arg1, arg2, and result
    - Triples: three fields
      - Operator, arg1 and arg2
    - DAG and Tree: similar to triples
    - Indirect Triples: pointers instead of position to store results

Compiler Design

- Intermediate Code Generation
  - Implementations
    - Quadruples
    - Triples
    - DAG and Tree
    - Indirect Triples

Implementations of 3-Address Code

3-address code

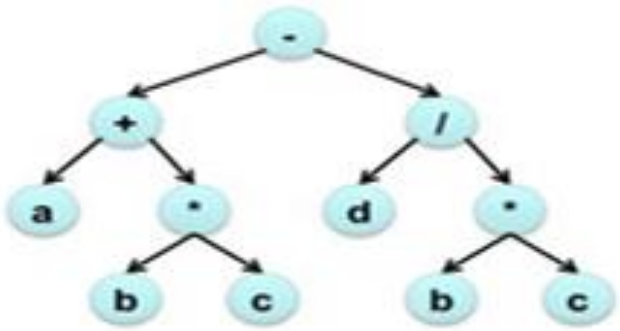
1. $t_1 = b*c$
2. $t_2 = a + t_1$
3. $t_3 = b*c$
4. $t_4 = d/t_3$
5. $t_5 = t_2-t_4$

Quadruples

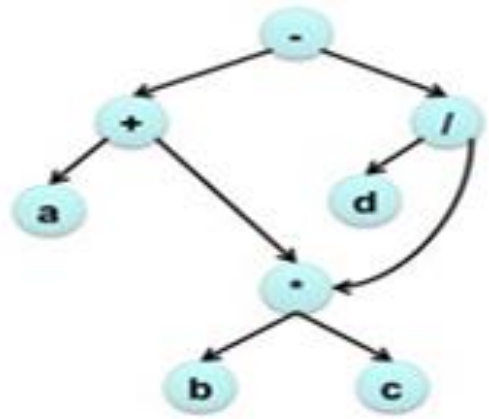
op	arg_1	arg_2	result
*	b	c	t1
+	a	t1	t2
*	b	c	t3
/	d	t3	t4
-	t2	t4	t5

Triples

	op	arg_1	arg_2
0	*	b	c
1	+	a	(0)
2	*	b	c
3	/	d	(2)
4	-	(1)	(3)



Syntax tree



DAG

Compiler Design

- Code Optimization
  - Remove redundant code without changing the meaning of program
  - Techniques
    - Common sub-expression elimination: Repeated appearance computed previously
    - Strength reduction: Replacement of expensive expressions with simple ones
    - Code movement: Moving a block of code outside a loop
    - Dead code elimination: Eliminate statements that are either never executed or unreachable

Compiler Design

- Code Optimization
  - Register Allocation
    - variables that die after being used only once can be reassigned for other expressions
    - Example:
      - a is only used in e's expression, e is only used in f's… name both a and e to same variable(register)

$$a = c + d$$
$$e = a + b$$
$$f = e - 1$$

$$r1 = r2 + r3$$
$$r1 = r1 + r4$$
$$r1 = r1 - 1$$

Compiler Design

- Code Optimization
  - Peephole Optimization: Transforming to optimal sequence of instructions
  - Common Techniques:
    - Elimination of redundant loads and stores
      - r2 = r1 + 5 ⇒ r3 = r1 + 5
      - I = r2
      - r3 = I
    - Constant folding
      - r2 = 3 * 2 ⇒ r2 = 6
    - Constant Propagation
      - r1 = 3 ⇒ r2 = 3 * 2 ⇒ r2 = 6 r2 = r1 * 2
    - Elimination of useless instructions
      - r1 = r1 + 0
      - r1 = r1 * 1